

Project Title Here

Scalable Data Engineering Solution

Data Engineering I (1TD169) – Spring 2026

Uppsala University
Department of Information Technology

Team Members: Group 32

Abdur Rehman Khalid
Arnab Kumar Ghosh
Dip Chowdhury
Muhammad Umair
Pradip Kumar Das
Zihao Yang

March 15, 2026

Contents

1	Background	3
2	Data Format & Characteristics	4
3	System Architecture & Implementation	5
3.1	System Design	5
3.2	Technology Stack	5
3.3	Infrastructure Configuration	6
3.4	Data Pipeline	6
3.5	Code Structure	6
3.6	Challenges & Solutions	7
4	Scalability Experiments & Results	8
4.1	Experimental Design	8
4.2	Horizontal Scaling Results	8
4.3	Vertical Scaling Results	8
4.4	Speedup & Efficiency Analysis	8
4.5	Bottleneck Identification	8
5	Discussion & Conclusion	9

1 Background

[Describe the scientific/application context of dataset. Explain the domain, why this dataset is important or interesting, what analyses others have performed with it (cite 2–3 relevant papers/resources), and state specific analysis objective.]

2 Data Format & Characteristics

[Describe the dataset technically: format (CSV, JSON, Parquet, etc.), schema/structure, total size and number of records, data quality issues encountered, and justification for format choice or conversion.]

3 System Architecture & Implementation

This section details the system architecture, infrastructure deployment, and technical implementation of our robust data pipeline. We outline the hardware topology deployed on OpenStack, why we selected our core software stack, and the engineering mechanisms driving our ETL and analysis jobs.

3.1 System Design

Our system follows a classic centralized-master, distributed-worker model tailored for batch processing big data. As depicted in Figure 1, the architecture is divided into three primary layers: Ingestion, Distributed Storage (HDFS), and Distributed Processing (Apache Spark). Data is fetched externally into the master node, pushed to HDFS, and subsequently processed by Spark executors dispersed across the worker nodes.

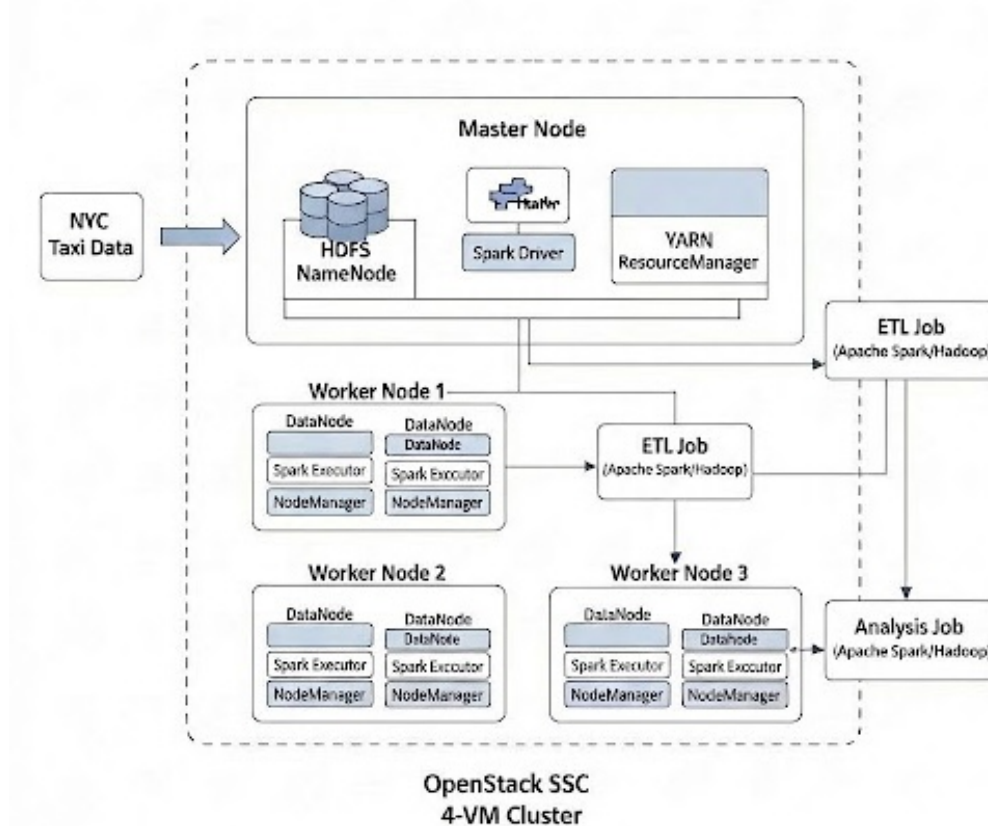


Figure 1: System architecture diagram showing the OpenStack deployment, node topology, and the internal data pipeline flow from raw Parquet to aggregated results.

3.2 Technology Stack

Given the project requirements, we adopted a modern big data stack focused on in-memory processing capabilities:

- **Storage (HDFS via Hadoop 3.4.1):** Chosen for its fault tolerance and native integration with Spark. While cloud object storage (like AWS S3) is popular today, deploying HDFS on bare VMs offers direct control over data locality and block replication factors.
- **Processing (Apache Spark & PySpark 3.5.1):** We selected Spark over standard Hadoop MapReduce because Spark holds intermediate RDDs and DataFrames in mem-

ory, drastically accelerating the iterative transformations required during our ETL phase. PySpark provides an accessible DataFrame API and straightforward type-casting structures.

- **Orchestration & Resource Management:** We relied on standalone cluster management managed heavily by Bash scripts (`run_benchmark.sh` and `deploy.sh`) combined with Git for CI/CD, creating a lightweight yet effective pipeline orchestration.

3.3 Infrastructure Configuration

Our cluster is deployed entirely on the Swedish National Infrastructure for Computing (SNIC) OpenStack instance. The infrastructure consists of 4 tightly coupled Virtual Machines (VMs):

- **Master Node (1x ssc.medium):** 2 vCPU, 4 GB RAM. This node is assigned a Floating IP for external SSH access. It houses the Hadoop NameNode, ResourceManager, and the Spark Driver process.
- **Worker Nodes (3x ssc.medium):** 2 vCPU, 4 GB RAM each. These sit on a private sub-network (192.168.2.x subnet). They run the Hadoop DataNodes, NodeManagers, and Spark Executor instances.

In total, the cluster harnesses 8 vCPUs and 16 GB of RAM, capped with a strict 100 GB collective storage limit. To maintain data reliability without exceeding storage quotas, HDFS was configured with a replica factor of 2 (`dfs.replication=2`).

3.4 Data Pipeline

The pipeline executes in three synchronized stages:

1. **Ingestion:** Raw NYC Taxi Parquet files are downloaded from the TLC portal to the Master node via `wget` and deposited straight into `hdfs://.../data/nyc-taxi/`.
2. **ETL Phase:** The `etl_job.py` script ingests the raw data block by block. It filters out invalid records (e.g., zero-distance trips, null metrics), converts timestamps to calculate `duration_minutes` and `pickup_hour`, aggressively prunes dropped columns, and commits the clean records back to HDFS at `/data/nyc-taxi-cleaned/`.
3. **Analysis Phase:** Finally, `analysis_job.py` aggregates the validated dataset. It groups by `PULocationID` and `pickup_hour` to compute the average fare and trip duration. It subsequently performs a left join against a static CSV (`taxi_zone_lookup.csv`) to convert abstract ID numbers into human-readable zone names, coalescing the final output into a single file for stakeholder ingestion.

3.5 Code Structure

The codebase follows a modular design paradigm:

- **src/config.py:** Centralizes cluster endpoints, absolute HDFS URI paths, and basic threshold configurations to eliminate hardcoded parameters across jobs.
- **src/etl_job.py:** The heaviest processor. Given resource constraints, it bypasses standard `spark.read.parquet("*.parquet")` schema inferences and iteratively targets singular files using `subprocess.run` to interact with HDFS.
- **src/analysis_job.py:** Houses the PySpark SQL aggregations, strictly outputting to a targeted CSV path using `coalesce(1)`.

- **scripts/run_benchmark.sh**: A specialized harness that automates execution over varied core allocations (`-total-executor-cores`) and node arrays to capture timestamps for plotting horizontal and vertical scales.

3.6 Challenges & Solutions

We encountered several prominent technical challenges during development:

1. **Dataset Schema Volatility**: When reading multi-month TLC data, PySpark's vectorized Parquet reader routinely crashed (`[CANNOT_MERGE_SCHEMAS]`) due to invisible shifts in the column type for `Airport_fee` (varying from `INT32` to `FLOAT64` dynamically by month). *Solution*: We restructured the ETL job into a `for`-loop that asks HDFS for the explicit list of file paths. Spark reads them independently, safely infers local schemas, selectively prunes the offensive column, and appends into a normalized structure.
2. **Strict Storage Exhaustion**: 25 GB of storage per VM meant HDFS was easily overwhelmed caching uncompressed RDDs and saving multiple layers of Parquet processing data. *Solution*: We established an aggressive "select and drop" mechanism in the ETL. We discard roughly 15 unused columns representing several gigabytes of overhead, saving only the 4 tightly packed elements (`PULocationID`, `fare_amount`, `pickup_hour`, `duration_minutes`) needed for analysis.
3. **Dynamic Allocation Skewing Benchmark Constraints**: Spark's dynamic resource allocation skewed the early iterations of our benchmarking by automatically distributing workloads across nodes we were attempting to restrict. *Solution*: We disabled dynamic allocation directly via PySpark config keys (`spark.dynamicAllocation.enabled=false`) and enforced hard caps using Spark submit metrics.

4 Scalability Experiments & Results

[Present experimental findings: experimental design, results with plots, analysis of results, bottleneck identification, and comparison to theoretical expectations.]

4.1 Experimental Design

[Describe what you varied in experiments (number of workers, cores, data size), the metrics measured (runtime, throughput), and the methodology used.]

4.2 Horizontal Scaling Results

[Present results of horizontal (strong) scaling experiments: fixed dataset size with increasing number of worker nodes. Include runtime vs. number of workers plot.]

4.3 Vertical Scaling Results

[Present results of vertical scaling experiments: fixed number of nodes with increasing cores/memory per node. Include runtime vs. resources plot.]

4.4 Speedup & Efficiency Analysis

[Compute and plot the speedup factor ($S = T_1/T_N$) and efficiency ($E = S/N$). Compare against ideal linear speedup. Discuss deviations.]

4.5 Bottleneck Identification

[Identify what limits scalability: CPU, network I/O, disk I/O, shuffle overhead, data skew, or serialization. Reference Amdahl's Law if applicable.]

5 Discussion & Conclusion

[Reflect on the project: Was the architecture suitable? What worked well? What would you improve with more time/resources? What did you learn about distributed systems?]